Longest Increasing Path in a Matrix (hard)

Problem Statement

Given a 2D matrix of size `m x n`, return the length of the `longest increasing path` in the matrix. The increasing path should have each element should be greater than the previous element in the path.

It is given that you can move only in 4-directions: `left`, `right`, `top`, and `bottom.`

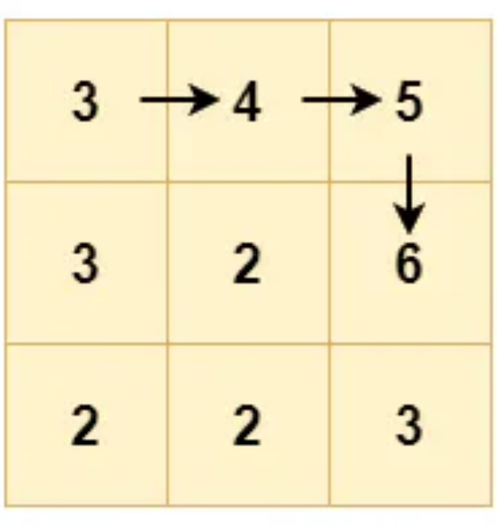
Examples

Example 1:

- **Input**: matrix =

```
[[3, 4, 5],
 [3, 2, 6],
 [2, 2, 1]]
```

- **Expected Output**: `4`



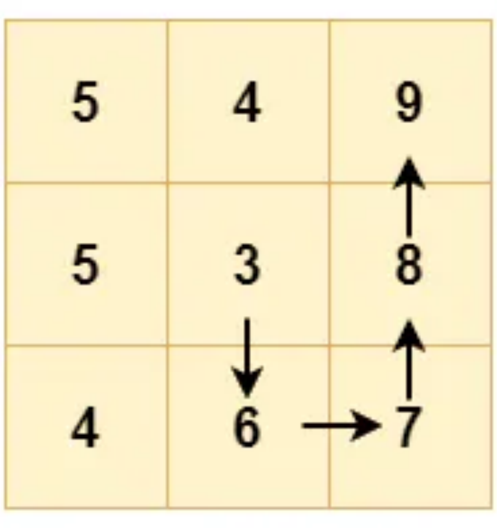
- **Justification**: The longest increasing path is `3 -> 4 -> 5 -> 6`. No other path has a greater length.

Example 2:

- **Input**: matrix =

```
[[5,4,9],
 [5,3,8],
 [4,6,7]]
```

- **Expected Output**: `5`



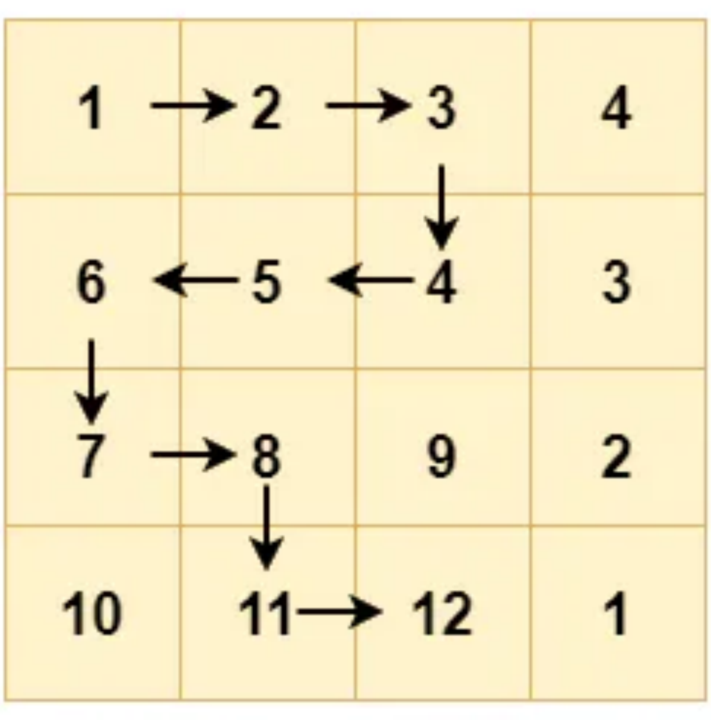
- **Justification**: The longest increasing path starts from the cell with value 3 (1, 1), moving to 6, then 7, 8, and finally 9.

Example 3:

- **Input**: matrix =

```
[[1, 2, 3, 4],
 [6, 5, 4, 3],
 [7, 8, 9, 2],
 [10, 11, 12, 1]]
```

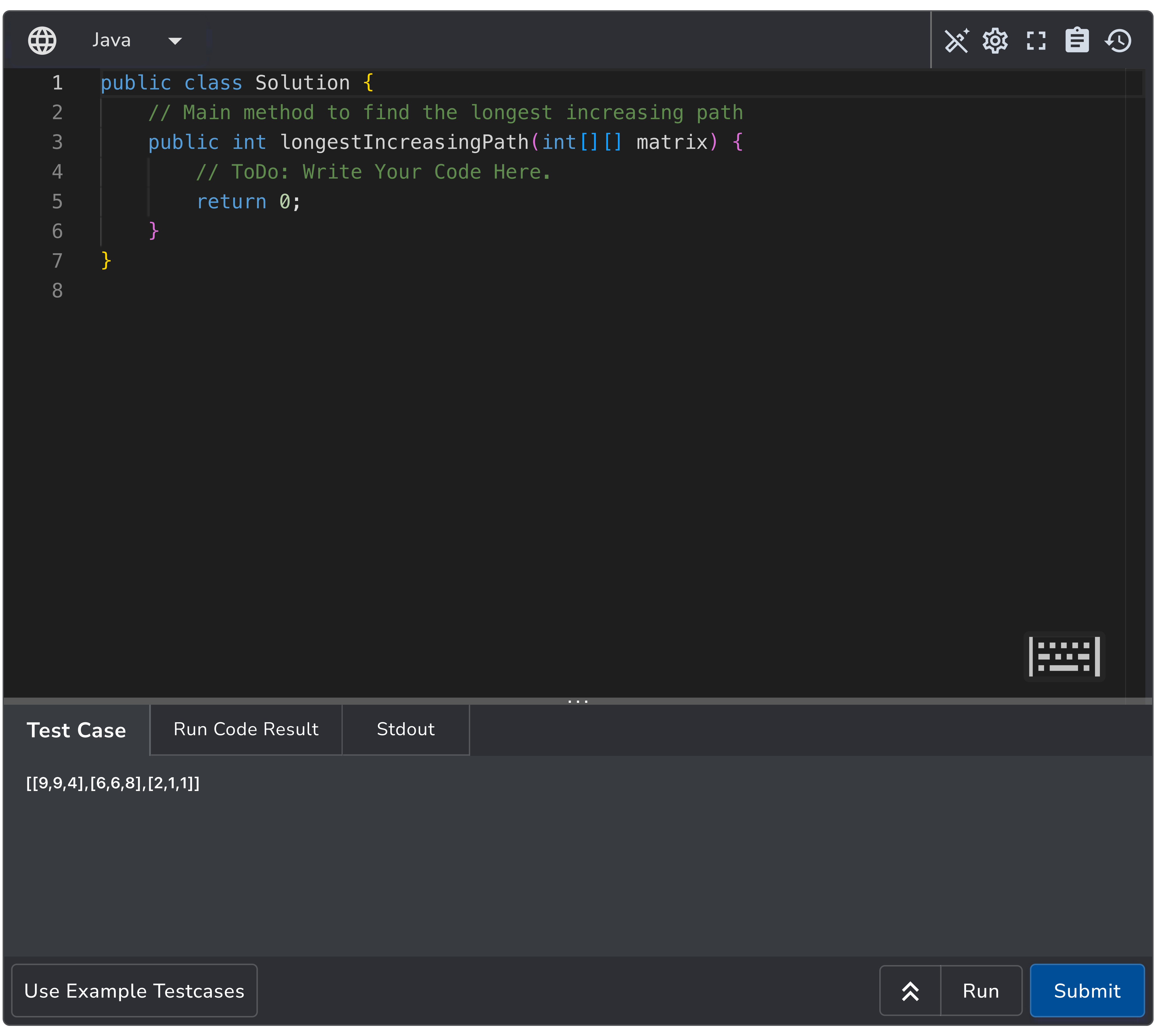
- **Expected Output**: `10`



- **Justification**: The longest increasing path is `1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 11 -> 12`.

Try it yourself

Try solving this question here:



```java
public class Solution {
    // Main method to find the longest increasing path
    public int longestIncreasingPath(int[][] matrix) {
        // ToDo: Write Your Code Here.
        return 0;
    }
}
```

[[9,9,4],[6,6,8],[2,1,1]]

← Previous
Solution: Serialize And Deserialize Binary Tree

Next →
Solution: Longest Increasing Path In A Matrix

☐ Mark as Completed